Rule-Based Systems

Rule Based Systems- Introduction

- Well-formed formula in propositional logic or FOL represent assertion knowledge
- Such wffs are divided into two categories
  - Rules: Assertion given in implication form
  - Facts: assertions that represent domain specific knowledge

- In logic we represent knowledge in a declarative and static way – as some facts and rules that are true
- Rules in logic say what is TRUE given some condition
- Rule-based systems are based on rules that say what to do, given various conditions
- A special interpreter controls when rules are invoked

- Simple examples are very similar in rules in logic
- However, in rule based systems we consider
- Other kinds of actions (apart from adding facts)
- Degree of certainty associated with facts
- Various different control schemes (not necessarily related to idea of logical proof)

Example Rules

Modus Ponens and Rule-Chaining

- A system whose KB is represented as a set of rules and facts is called a Rule Based System
- A rule-based system consists of a collection of IF-THEN rules, a collections of facts and some interpreter controlling the application of the rules, given the facts

- Rules are represented in the following form:
- IF <antecedent> THEN <consequent>
- When the antecedent part is NULL then Rule becomes a fact
- Rules are normally represented as HORN clause

HC

- Horn clauses can have at most one non-negative clause

- Triggered and fired rules
- A rule is triggered when all the antecedents evaluate to true
- A rule is fired when the action stated in the consequent part of the inference related to the consequent part is inferred

Rule-based system architecture

Inference Mechanism

- A machine that implements strategies to utilise the KB and derive new conclusion from it
- Automata of IM (is the study of mathematical object)

- The execute state fires the rules once all its antecedents match
- essentially, the function of the execute state can be thought of as searching a path to the goal in a search space

Example

Backtracking

Conflict resolution

- The objective is to decide which of the triggered rules in a particular stage should be fired
- Can use various strategies
  - FCFC (first come first serve) (rule ordering
  - Specificity ordering
  - Fire all
  - Heuristic measure (distance from the goal)

example

Conflict resolution strategies

- Refractoriness: Rules once fired will not be fired later
- Meta-rules: rules about rules embedded within the inference machine that provide the information about which of the rules apply under what conditions.
- 90% of computing time is spend in the match phase, hence minimizing the same is necessary

Searching the space

- Start from the given facts try to arrive at the goal [data-driven]
- Start from the goal and try to prove the goal using the given facts [goal-driven]

- In the case of rule-based system how is the search space defined?
- Example???

- Reasoning mechanism in rule-based systems

Forward chaining mechanism (data-driven)

- Starting from the start state applying rules one by one to arrive at the goial state
- Also known as data-driven search
- Forward chaining may lead search to a dead end. In such cases backtracking is necessary [give example]
- Backtracking strategies can be
  - Chronological
  - intelligent

- In Fc system facts are held ina WM
- Copndition action rules represent actions to take when speciofied facts occur in WM
- Typically the actions involve adding or deleting facts from WM
- control

Forward Chaining

- Control cycle called recognise –act cycle
- Repeat
  - Find all rules which have satisfied conditions given facts in working memory
  - Choose one, using conmflict resolution strategy
  - Perform action in conclusion, probably modifying WM
- Until no rules can fire, or halt symbol added to WM

Example

- Simple “fire” example from earlier
- Working memory initially contains
  - Alarm-beeps
  - Hot
- Following the algorithm: first cycle
  - Find all rules with satisfied (match)
  - Condition: R2
  - Choose one R2
  - Perform action: ADD smokes
  - Working memory contains
    - Alarm-beeps, hot, smokey

- Continue next cycle and write yourself

Forward Chaining Applications

- Forward chaining systems have been used as:
  - A model of human reasoning
  - Basis for expert systems: various expert system shells based on this model, such as CLIPS
  - Practical forward chaining systems supportr pattern matching
  - Example CLIPS rule:
  - (default fire-alarm
  - (temperature? R1 hot
  - environment? R2 smoky)
  - =>
  - (assert (fire-in? r1))

Architecture

Backward chaining

- Some rules/facts may be processed differently, using backward chaining interpreter
- This allows rather more focused style of reasoning (forward chaining may result in a lot of irrelevent conclusions added in working memory/)
- Start with possible hypothesis, Should I switch the sprinklers on?
- Set this as a goal to prove
  - Similar to Prolog which uses a backward chaining style of reasoning

Backward chaining

- Basic algorithm
- To prove goal G:
  - If G is in the initial facts, it is proven
  - Otherwise, find a rule which can be used to conclude G and try to prove each of that rule's condition

example

Backward chaining example

- Should we switch_on the sprinklers? Set as a goal:
  - G1: switch_on_sprinklers
  - Is it the initial facts? No, is there a rule which adds this as a conclusion? Yes, re
  - Set condition of r3 as new goal to prove
  - G2: fire
  - Is it in initial facts: No, Rule? Yes, r1
  - Set conditions as new goals: G3: hot, G4 smokey

- continue

Examples of RBS

- Available expert system tools
  - CLIPS
  - NEXPERT
- Language facilitating development of Rule Based Systems by incorporating within in structure certain advanced storage techniques
  - PROLOG

Expert systems

- Systems acting in a particular domain and behaving like human expert therein
- Normally expert systems are applied to problems for which no algorithmic solution exist
- Guided by domain sp[ecific rules and should be able to explain irts actions

ES applications

- Rule Based Systems have been widely used in expert systems
- E.g. medical systems, where start with set of hypothesis on possible disease and try to prove each one, asking addittional nquestions of using when fact is known
  - MYCIN pioneering

Vehicle diagnostic system

R1: If gas_in_engine and turns_over, then problem (Spark_plugs)
R2: If not(turns_over) and not (lights_on) then problem (battery)
R3: If not (turns_over) and lights_on, then problem (starter)
R4: If gas_in_turn and gas_in_carb then gas_in_engine

Facts
Known: gas_in_tank
Known: gas_in_carb
Known: gas_in_engine
Known: turns_over
X = spark_plugs (by R1)

Solve this using FC and BC
Supose the goal is to prove problem (battery) with the same set of initial facts. Draw the search space based on the rules fired.

Frame

What is Frame?

- A frame is a prototype of a concept
  - Denoting the attributes of the concept
  - The class of objects or concepts to which the concept in question..
  - And some more things
- An instance of a frame is a representation of a SPECIFIC object
- Instantiation of a frame

Example

- Fido is a dog
- The concept dog
- Fido is a particular dog
- How can we represent the concept/notion of the creature “dog”?

- Frame consists of number of slots

Economy of represenation

Object attribute value

- An instance of a frame denotes an object
- A frame denotes a class to which the object belongs
- The slots denote the attributes of the object
- The slots can be of different types
- The slots are instantiated with values

slots

- Slots denote attributes
- Attributes are typed
- Some values are defined in the frame definition
- Some values are defined in the instances only
- Instantiated slots define facts may be used to answer queries

What are there in a slot

- Values
- Types
- Constraints over possible values
- Predicates
- Comapre with object oriented systems
- Slots can be structured frames within slots pointers frames

Inferencing in a Frame

- Generic values
- Default value
- Multiple inheritence